String matching

Find every occurrence of a pattern in a text, including overlaps. Compare direct search, rolling hashes, finite automata and KMP.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A roadmap for matching

Define the output and edge cases first. Then trace filtering, state transitions and prefix fallback; distinguish correctness from running time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

String matching

Text T has length n and pattern P has length m. Report every zero-based s with T[s:s+m]=P, including overlapping matches.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Boundary contract

The empty pattern matches all n+1 boundaries 0..n. A nonempty pattern longer than the text has no matches. The remaining algorithms first assume $1 \le m \le n$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Four methods

Naive compares directly. Rabin–Karp filters by hash. A DFA precomputes transitions. KMP uses prefix links to fall back on demand.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Naive matching

For every s=0..n−m compare P[j] with T[s+j] in order. Report s if all match. Increment s by one to retain overlaps.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Naive trace

T=ababa, P=aba. At s=0 compare a,b,a and match; s=1 fails at b≠a; s=2 compares a,b,a and matches. Output 0,2.

Naive matching

```
T   a b a b a
P   a b a     → 0
P     a b a   → fail
P       a b a → 2
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Naive cost

At most (n−m+1)m comparisons give worst-case O((n−m+1)m) time and O(1) extra space. Repeated characters can approach the worst case.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rabin–Karp fingerprints

For symbols in 0..d−1, hash a length-m string as Σ x[j]d^(m−1−j) mod q. Equal strings have equal hashes; the converse is false.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rolling hash

Let h=d^(m−1) mod q. Remove a and append b using t′=(d(t−a h)+b) mod q. Normalize negative remainders into 0..q−1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deriving the rolling update

Let W=T[s:s+m], and let c=T[s+m] be the next symbol. Treat symbols as values and write the unreduced polynomial H(W)=Σ(j=0..m−1) W[j]d^(m−1−j).

Remove the high term W[0]d^(m−1), multiply by d, and add c:

H(T[s+1:s+m+1]) = d(H(W)−W[0]d^(m−1))+c.

Each remaining term shifts one position. Reduce modulo q; with t_s=H(W) mod q and h=d^(m−1) mod q, t_(s+1)=(d(t_s−W[0]h)+c) mod q. Any q≥2 works; primality is unnecessary here.

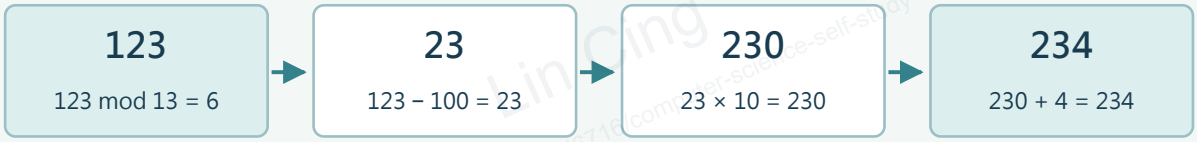
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rolling-scan loop invariant

Before processing shift s, t_s is the hash of the current window T[s:s+m], and p is the pattern hash. Horner's rule initializes p and t_0. If s<n−m, the identity on the preceding slide establishes the invariant for s+1; the last window does not read T[n].

Equal strings have equal hashes, so no true match is filtered out. Verify characters on equal hash, so collisions cannot cause false reports. An intermediate difference may be negative: normalize the final remainder to 0..q−1.

d=10, q=13, m=3: window 123 → 234



h=100 mod 13=9; (10·(6−1·9)+4) mod 13=0.
Normalize negative remainders; 234 mod 13=0.

Rolling hash trace

With d=10, q=13, m=2, T=1234 and P=34, window hashes for 12,23,34 are 12,10,8. The pattern hash is 8; confirm the final window character by character.

Slide the window one position



remove 1 → ×10 → append 3
(10·(12 − 1·10) + 3) mod 13 = 10

remove 2 → ×10 → append 4
(10·(10 − 2·10) + 4) mod 13 = 8

h = 10; reducing an intermediate value modulo 13 preserves the result.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Collisions and cost

For $1 \le m \le n$ and constant-time modular arithmetic, let H count all equal-hash windows, including true matches. Verification gives $O(n+m+mH)$, with worst case $\Theta(n+m+m(n-m+1))$.

If $T=a^n$ and $P=a^{\lfloor n/2 \rfloor}$, every window is a true match and verification costs $\Theta(nm)$, even with no false hits.

If a randomized hash gives each unequal window collision probability at most $1/q$, and V is the number of true matches, expected time is $O(n+m+mV+mn/q)$. Near-linear time also requires bounded true-match verification, for example $V=O(1)$ and $q \ge m$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Random-base hashing: assumptions

Fix the input first, then choose b uniformly from the finite field F_q for prime q. Distinct symbols have distinct encodings in F_q. For a length-m string X, H_b(X)=Σ(j=0..m−1) X[j]・b^(m−1−j); use b⁰=1 even when b=0.

For fixed distinct X,Y, D(b)=H_b(X)−H_b(Y) is a nonzero polynomial of degree at most m−1. A nonzero field polynomial has at most its degree many roots, hence collision probability ≤min(1,(m−1)/q), not unconditionally 1/q.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Root bound and expected running time

If r is a root, factor D(x)=(x−r)Q(x). Every other distinct root is a root of Q; repeating the argument bounds the number of roots by the degree.

Let N=n−m+1 and z be the number of true matches. Linearity of expectation over the nonmatching-window indicators needs no independence between collisions: E[f]≤(N−z)min(1,(m−1)/q). With character-by-character verification, expected time is O(n+m+mz+m(N−z)min(1,(m−1)/q)), assuming O(1) field operations. The mz verification term for true matches remains.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One false hit and two true matches

Use digit encodings, q=11, b=10, m=2, P=12, T=122312. $H_b(xy)=10x+y \bmod 11$; the pattern hash is 1.

The five windows 12, 22, 23, 31, 12 have hashes 1, 0, 1, 9, 1. Window 23 has the same hash as P but differs; character verification rejects it. Positions 0 and 4 are true matches.

q=11, b=10: equal hashes still require verification

s = 0	s = 1	s = 2	s = 3	s = 4
12 → 1	22 → 0	23 → 1	31 → 9	12 → 1
match	skip	false hit	skip	match

P=12 hashes to 1; verify, output 0 and 4, reject 23.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exact collision probability in the example

Now choose b uniformly from F_{11} rather than fixing b=10. The collision equations for nonmatching windows 22, 23, 31 against P=12 have the respective solutions b=0, b=10, b=6; each collides with probability 1/11.

With collision indicators I_1, I_2, I_3, $E[f]=E[I_1]+E[I_2]+E[I_3]=3/11$. No independence is needed. At b=10, only 23 is a false hit, and both true matches still require verification.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Automaton state

State q is the length of the longest suffix of the scanned text that is also a prefix of P, with $0 \le q \le m$. After $\delta(q,c)$, report $end-m+1$ when $q=m$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

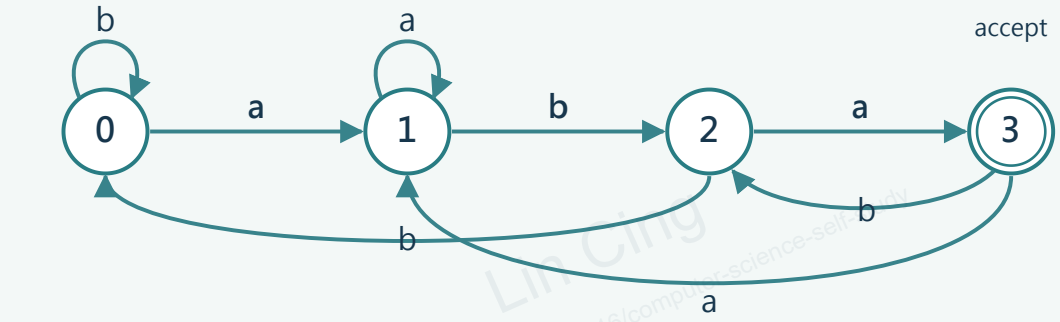
DFA transition definition

$\delta(q,c)=\max\{k \le m: P[:k] \text{ is a suffix of } P[:q]+c\}$. Include transitions out of accepting state m so overlapping matches are found.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DFA transition table

For P=aba and alphabet {a,b}, transitions (a,b) are 0:(1,0), 1:(1,2), 2:(3,0), 3:(1,2). Scanning takes O(n); the table uses O(m|Σ|) space.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Automaton state as a longest suffix

Let P⟨k⟩ be the pattern prefix of length k, with P⟨0⟩=ε; P[i] denotes the zero-based character at index i. σ(x) is the length of the longest pattern prefix that is a suffix of x. State q=σ(x), and δ(q,a)=σ(P⟨q⟩a).

The key identity is δ(σ(x),a)=σ(xa): the state suffices to update the match without storing the entire text prefix x.

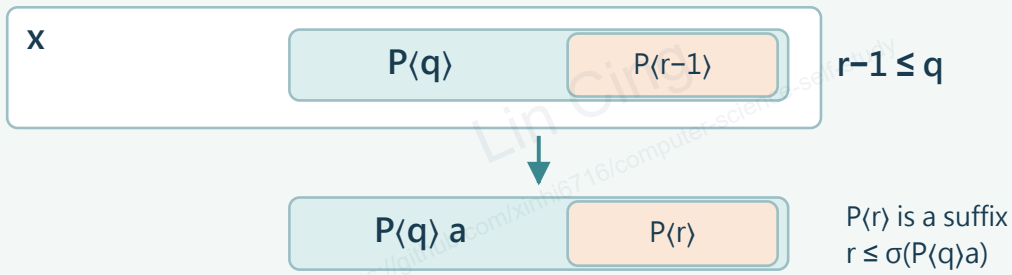
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

State identity: the first inequality

Let $q=\sigma(x)$, $r=\sigma(xa)$. If $r>0$, removing the last character of $P\langle r\rangle$ shows that $P\langle r-1\rangle$ is a suffix of x, so $r-1\le q$.

Both $P\langle q\rangle$ and $P\langle r-1\rangle$ are suffixes of x; the shorter is a suffix of the longer. Thus $P\langle r-1\rangle$ is a suffix of $P\langle q\rangle$, and after appending a, $P\langle r\rangle$ is a suffix of $P\langle q\rangle a$. Hence $r\le\sigma(P\langle q\rangle a)$. For $r=0$ the inequality is immediate.

Compare suffixes of x, then append the same symbol a



State identity: converse and correctness

Since $P\langle q\rangle$ is a suffix of x, $P\langle q\rangle a$ is a suffix of xa; every suffix of $P\langle q\rangle a$ is also a suffix of xa. Thus $\sigma(P\langle q\rangle a)\le\sigma(xa)=r$. Together the inequalities give equality.

Initially $\sigma(\varepsilon)=0$. Induction on the input establishes that the state is σ(the text read so far). State m occurs exactly when P is a suffix of the scanned text, which is exactly a match. Continue transitions after acceptance to detect overlap.

Build the DFA using π

Let π[q−1] be the length of the longest proper border of P⟨q⟩. For finite alphabet Σ and m≥1:

• q=0: δ(0,a)=1 if a=P[0], and 0 otherwise. • 1≤q≤m: if q<m and a=P[q], set δ(q,a)=q+1; otherwise set δ(q,a)=δ(π[q−1],a).

At q=m do not read P[m]: use the second branch. Since π[q−1]<q, the required row has already been computed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why border fallback builds the table

When direct extension fails, a match of length q+1 is impossible. For any shorter nonempty match P⟨k⟩, dropping its last character leaves a proper border P⟨k−1⟩ of P⟨q⟩. All proper borders occur on the π chain, so retrying the same a from the longest proper border loses no candidate; k=0 remains possible.

Computing π uses O(m) time and space. There are m+1 rows and |Σ| constant-time cells per row: table time and space O(m|Σ|), assuming O(1) alphabet and table access. Scanning costs O(n) plus output.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prefix function

$\pi[i]$ is the length of the longest proper prefix of $P[:i+1]$ that is also its suffix, with $0 \leq \pi[i] \leq i$. Proper excludes the whole string.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prefix function example

For P=ababaca at indices 0..6, π=[0,0,1,2,3,0,1]. Symbols a b a b a c a correspond to 0 0 1 2 3 0 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Build the prefix function token by token

i=0:a→0; 1:b differs from a→0; 2:a=a→1; 3:b=b→2; 4:a=a→3; 5:c≠b, 3→1, c≠b, 1→0, c≠a→0; 6:a=a→1.

i = 5: repeated fallback while comparing c

index	0	1	2	3	4	5	6	
	a	b	a	b	a	c	a	
k = 3			a	b	a	b ≠ c		3 → 1
k = 1					a	b ≠ c		1 → 0
k = 0						a ≠ c		π[5] = 0

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why fallback works

If k characters match but the next does not, a shorter candidate must be a border of P[:k]. The longest candidate is π[k−1]. Repeat until a match or k=0. For P=ababaca, k=5 and next symbol x, fallback is 5→3→1→0: one fallback is insufficient.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why the border chain is complete

Let B be the longest proper border of X. Any shorter proper border C is both a prefix and a suffix of X, hence a prefix and a suffix of B, and therefore a border of B. Conversely, any border of B is a border of X.

Repeatedly apply π[length−1], beginning at |X|. Length strictly decreases to 0; the two containments prove that the chain lists exactly all proper border lengths, without omissions.

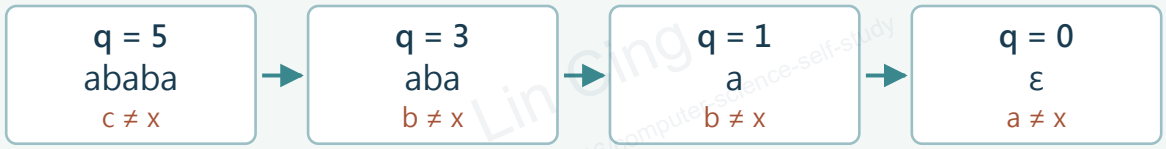
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prefix function: invariant and proof

If m=0, π is empty. Otherwise set π[0]=0: one symbol has no nonempty proper border. For i=1,…,m−1, known earlier entries give k=π[i−1]. While k>0 and P[k]≠P[i], set k←π[k−1], keeping P[i] fixed.

Candidates are visited from longest to shortest. On equality set π[i]=k+1; if k=0 and P[0] differs, set π[i]=0. This completes the induction.

P = ababaca: retry the same symbol x



5 → 3 → 1 → 0; at q=0 compare P[0], then stay at 0.

Every shorter border is a border of the longest one.

Why π takes linear time

Use the nonnegative potential k. Each i can increase k at most once, by at most one; each while fallback lowers k by at least one. Starting at zero, total fallbacks cannot exceed total increases $\leq m-1$.

Each i also performs at most one terminating comparison and constant work. Thus preprocessing is O(m); the same argument gives O(n) for the KMP scan. This bounds the entire run, although one input character can trigger multiple fallbacks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

KMP preprocessing

Set $\pi[0]=0$. For $i=1..m-1$, start $k=\pi[i-1]$; while $k>0$ and characters differ set $k=\pi[k-1]$; increment k on equality; set $\pi[i]=k$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

KMP scan

Start $q=0$. For each c, while $q>0$ and $P[q] \neq c$ set $q=\pi[q-1]$. Increment q on equality. At $q=m$ report $i-m+1$, then set $q=\pi[m-1]$ to preserve overlaps.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two KMP scan states

Assume $m \geq 1$. Let $\sigma(x)$ be the longest pattern-prefix suffix of x, allowing length m; let $\rho(x)$ be the longest such prefix with length strictly less than m.

At the start of each iteration $q=\rho(x)$, where x is the text read so far; initially $q=\rho(\varepsilon)=0$. If $\sigma(x) <m$, then $\rho(x)=\sigma(x)$. If $\sigma(x)=m$, then $\rho(x)=\pi[m-1]$. Thus the stored post-match state is the longest proper border, not m.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A scan step simulates the DFA

For the same new symbol a, the while loop tries the border chain from q in decreasing order. The first extendable candidate is longest; if none extends, the state is 0. Thus the temporary state $r=\delta(q,a)$.

If $\sigma(x)<m$, $q=\sigma(x)$; if $\sigma(x)=m$, $q=\pi[m-1]$ and $\delta(m,a)=\delta(\pi[m-1],a)$. By $\delta(\sigma(x),a)=\sigma(xa)$, in both cases $r=\sigma(xa)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why post-match fallback preserves overlaps

$r=m$ exactly when the pattern is a suffix of the scanned text xa, so output $i-m+1$ is correct. Then $q\leftarrow\pi[m-1]=\rho(xa)$. If $r<m$, set $q\leftarrow r=\rho(xa)$. Either way the next iteration's invariant is restored.

For P=aba and T=ababa, positions 2 and 4 produce outputs 0 and 2. After either acceptance, retain q=1; extending that border preserves the overlapping occurrence.

P=aba, T=ababa: accept first, retain the border

i	0	1	2	3	4
T[i]	a	b	a	b	a
q before	0	1	2	1	2
r = σ	1	2	3	2	3
q after	1	2	1	2	1
output	–	–	0	–	2

When r=3 report, then set $q=\pi[2]=1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

KMP token trace

T=ababa, P=aba, π=0,0,1. Each i, symbol, q: 0 a→1; 1 b→2; 2 a→3, report 0 and fall back to 1; 3 b→2; 4 a→3, report 2 and fall back to 1.

P = aba: retain border length 1 after a match

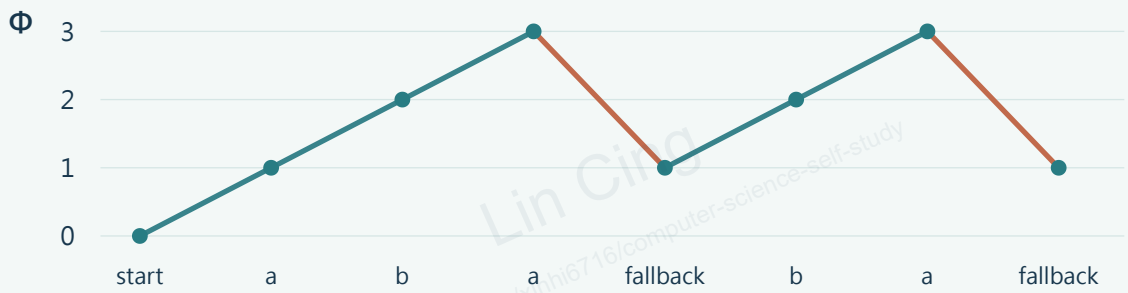
T index	0	1	2	3	4
input	a	b	a	b	a
state q	1	2	3	2	3
match at			0 3 → 1		2 3 → 1

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Amortized linear time

Use q or k as potential in each preprocessing or scan pass. Each input position increases it by at most 1; every nonzero fallback decreases it by at least 1. Total fallbacks cannot exceed total increases. Other comparisons cost O(1) per position, so total time is O(m+n).

Potential Φ = q: each input raises it at most once



Every fallback drops Φ by at least 1; total drops ≤ total rises.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

KMP time and memory

Preprocessing costs O(m), scanning O(n), and the π table O(m). Storing z outputs costs an additional O(z); streaming output avoids that storage.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Four methods compared

For $1 \le m \le n$, naive takes worst-case O(nm) time and O(1) space; Rabin–Karp worst-case O(nm) and O(1) workspace; DFA O(n) scan and O(m|Σ|) table; KMP O(n+m) and O(m).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Implementation pitfalls

Specify the symbol and index unit: bytes, Unicode code points, and grapheme clusters differ. Handle the empty pattern first; verify hash hits; do not reset to zero after a match.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Verification strategy

Use direct enumeration as an oracle and exhaust all short texts and patterns over a small alphabet. Compare complete outputs of Rabin–Karp, DFA and KMP, including collisions, empty patterns, m>n, and overlaps.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Takeaway

Borders of the matched prefix form the viable fallback chain. KMP follows it on demand; the DFA precomputes state updates; hashing requires verification to guarantee correctness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

CLRS, Introduction to Algorithms, 3rd ed., chapter 32, pp. 990–1006. MIT 6.046J Fall 2004, Lecture 22.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study